PROJECT INTERMIX

Clarity v1.2.1 Community Readiness and Daily Field Guide

A verified checkpoint for a private, fully local intelligence companion running on Android through Termux, LiteRT-LM, and an 8K bounded context architecture.

STATUS	FIELD TEST CANDIDATE
DEVICE	PIXEL 10 PRO / ANDROID 17
INFERENCE	GEMMA 4 E4B / GPU / 8,000 KV
VALIDATION	34 OF 34 PACKAGE TESTS PASSED

This document is a factual handoff, a safe community boundary, and a repeatable daily test protocol. It is not a public-release announcement or a claim of production readiness.

1. Verified checkpoint

Clarity v1.2.1 was installed on the target Pixel on 24 August 2026. The installer ran the packaged suite, verified device capabilities, created a rollback snapshot, and reported no new legacy-memory imports. The live acceptance screen then demonstrated exact cross-session recall and styled Markdown output.

34 / 34	8,000	v1.2.1	0
PACKAGE TESTS PASSED	PHYSICAL KV TOKENS	INSTALLED CLARITY BUILD	LEGACY ITEMS REIMPORTED

Acceptance evidence

- A fresh session answered the broad question about previous work with the exact file `continuity_probe.py`, the verified output `CONTINUITY_OK`, and the completed mission state.
- The same device rendered a heading, bullet list, bold emphasis, and inline code after streaming completed.
- The installed SQLite migration report found 12 known duplicates and 0 importable records, so conversation data was not changed.
- The exact rollback snapshot is `~/project-intermix/archive/pre_clarity_install_20260824_225805`.

Reference runtime

Layer	Verified configuration
Device	Google Pixel 10 Pro, Tensor G5, Android 17
Host	Termux with Python 3.13
Inference	LiteRT-LM 0.16.1, GPU/OpenCL, resident Engine
Model	Gemma 4 E4B LiteRT model
Context	8,000 physical KV tokens with adaptive virtual context
Memory	SQLite schema v1 with FTS5, sessions, durable facts, and summaries
Workspace	Fixed local workspace, ordered tools, tests, checkpoints, and deletion review

2. System behavior and trust boundaries

Capability	What is implemented	Authority boundary
Conversation memory	Recent turns, session checkpoints, retrieved durable memories	Model must not claim missing memory
Verified work recall	Relevant prior-work questions receive controller mission records	Files and tests come from executor state
Web grounding	Forced /web and intent-gated automatic checks	Current claims require relevant evidence
Workspace agent	Read, write, run Python, repair across fresh epochs	Fixed root; ordered tools; bounded execution
Deletion	Proposal and approval workflow	No automatic permanent deletion
Idle maintenance	Incremental docs and trusted-source research	Only while app is open and idle
Interface	Throttled plain streaming then one Markdown render	No per-token Markdown parsing

Community evidence policy

- Official repositories and vendor documentation may support implementation facts.
- Verified registries and signed releases may support package and version facts.
- Community issues and discussions are useful leads, not self-verifying instructions.
- A community workaround must be reproduced locally or corroborated before it becomes a recommended fix.
- General web results remain disabled for idle research. Explicit user web requests use the separate grounding path.
- Research output may propose a change but cannot execute code or expand its own permissions.

Current non-claims

Intermix is not yet a distributable Android product, an OS-level sandbox, an infallible memory system, or an autonomous updater. The current package is a strong developer build validated on one primary device profile. Multimodal input and public deployment logic remain future work.

3. Safe community package boundary

A public or tester-facing package must be assembled from source modules and clean templates. It must never be produced by copying the live device directory wholesale.

Include

- Engine source files, deterministic tests, installer, VERSION, README, release notes, and device acceptance guide.
- Empty database initialization and an empty workspace skeleton.
- A dependency manifest, supported-version matrix, rollback instructions, and known limitations.
- Sanitized example prompts, expected outcomes, and a reproducible checksum.
- License and attribution files after the distribution model is deliberately selected.

Exclude

- Model weights, compiled model caches, authentication tokens, and provider secrets.
- The live `sovereign.db`, legacy conversation text, identity lore, personal exports, and user-specific memory.
- Workspace project files, `.intermix` task ledgers, checkpoints, trash, research findings, and local Git metadata.
- Absolute Android paths, device identifiers, private screenshots, crash dumps, or logs that contain prompts and responses.
- Any self-generated claim that has not been confirmed by code, a device check, or authoritative documentation.

Before sharing any diagnostic bundle

Review every file manually, remove message content by default, replace home and storage paths with placeholders, retain only the minimum timing and error evidence, and show the exact bundle contents before export.

4. Daily field-test loop

Run a short, repeatable cycle rather than trying every feature every day. The goal is to discover drift, friction, and recurring failure patterns during normal use.

Step	Test	Pass signal
1	Cold launch and /status	Cockpit opens; schema and KV are correct; no crash
2	One natural everyday question	Relevant, concise answer; no irrelevant web warning
3	One longer explanation	Depth expands when requested; Markdown stays readable
4	One continuity question	Known prior fact or verified task is recalled accurately
5	One forced /web query	Relevant source-backed answer or explicit safe refusal
6	One bounded workspace task when useful	Ordered actions, real test output, verified completion
7	Restart and resume	Transcript, session, and mission state rehydrate
8	Record friction	Time, prompt, expected result, actual result, severity

Minimal daily record

```
Date: ____ Build: ____ Android: ____ Free RAM before: ____ Engine: cold / hot
Task: ________________________________________________
Expected: _____________________________________________
Actual: _______________________________________________
First visible text: ____ s Total: ____ s Grounding: skipped / passed / unavailable
Memory result: correct / incomplete / wrong Workspace result: N/A / passed / failed
Severity: P0 / P1 / P2 / P3 Reproduces after restart: yes / no
```

Severity guide

Level	Meaning	Response
P0	Data loss, unsafe deletion, privacy exposure	Stop testing; restore backup; preserve evidence
P1	Crash/OOM, infinite loop, verified recall becomes false	Capture sanitized status; reproduce once; investigate
P2	Slow response, irrelevant grounding, incomplete tool result	Log conditions and frequency; continue cautiously
P3	Spacing, color, wording, minor visual inconsistency	Collect for a later UX batch

5. Community issue and release protocol

Issue report template

```
Title: [P0-P3] concise symptom
Build and checksum: ___________________________________
Device / Android / Termux / LiteRT: ___________________
Engine state before request: cold / hot / fallback
Exact sanitized prompt or command: ____________________
Expected behavior: ____________________________________
Actual behavior: ______________________________________
Timing and available RAM: ______________________________
Reproduction steps: ___________________________________
Sanitized evidence attached: status / trace / screenshot / none
Private data review completed: yes / no
```

Recommended gates before a community beta

Gate	Acceptance condition
G1	Seven consecutive days of ordinary use with no data loss or unsafe workspace behavior.
G2	A written matrix of at least 25 mixed chat, recall, web, and workspace tasks with pass/fail outcomes.
G3	Clean installation and rollback on a second device or a fresh Termux environment.
G4	A deterministic privacy scrubber and an inspect-before-export diagnostic flow.
G5	Dependency, license, model-distribution, and third-party attribution review.
G6	Versioned release archive, checksum, changelog, known-issues list, and reproducible test command.

Next engineering horizon

- Keep v1.2.x focused on reliability evidence, search coverage, issue reproduction, and installer hardening.
- Treat self-audit and proactive diagnostics as proposal-only features with evidence, runtime estimates, scope, and approval.
- Add intent and preference weighting only after field logs show where the current policy repeatedly misclassifies requests.
- Defer multimodal and official deployment work until text, memory, web, and workspace behavior remain stable across devices.

End-of-day checkpoint

Clarity v1.2.1 is installed, rollback-protected, and accepted on its primary device. Daily field use is now the correct next experiment: observe whether it reduces friction and remains trustworthy during real tasks before expanding scope.